



KAFKA CONNECT

Kafka Connect Support and Services

Connector engineering. Platform expertise.
Support that fits your operating model.

SERVICE OVERVIEW

oso.sh

October 2026 | Version 1.1

Connectors with clear ownership

OSO supports its own connectors, all Debezium connector families and all Aiven Open Kafka Connect implementations. Each agreement records versions, environments and engineering responsibilities.

Salesforce and ServiceNow | Sources and sinks

OSO-maintained, Apache 2.0. Salesforce covers Pub/Sub CDC, Platform Events, Bulk API backfill and sink integrations. ServiceNow covers REST Table API ingestion and CRUD delivery. Published product policies define maintained releases and security obligations.

Oracle Database CDC | Upcoming OSO source

An OSO Oracle Database CDC connector is being prepared for open-source release. Contact OSO for support and adoption planning. Public release details and the supported deployment profile will be published with the connector.

All Debezium connector families

Commercial support covers documented source and sink families and incubating connector projects. This includes relational, document, distributed and vector database integrations. Assess snapshots, capture prerequisites, offsets and recovery against the exact release; maturity differs between projects.

All Aiven Open Kafka Connect connectors

Commercial support covers JDBC sources/sinks, HTTP, Elasticsearch, OpenSearch, BigQuery, S3 source/sink, GCS, Azure Blob Storage, Salesforce sources/sinks and AMQP source. Existing legacy S3, GCS and BigQuery deployments can also be supported with an agreed maintenance and migration route.

Full connector inventory and support scope

Every connector is listed with direction, upstream link and maturity in the public GitHub catalogue. Third-party engineering and security-maintenance responsibilities are agreed explicitly. Support is delivered by OSO, independently of an upstream managed-service subscription.

Full inventory: github.com/osodevops/kafka-connect-support-catalogue
Incubating, archived and upcoming projects are labelled in the catalogue.

Specialists alongside your operations team

Your team keeps control of production. OSO supplies Kafka Connect expertise and agreed engineering assistance through a clear incident and escalation process.

A practical incident workflow

Your operations team monitors the service and performs initial triage. Raise a ticket through the agreed support channel, share redacted diagnostics securely, and join an incident call where needed. OSO investigates, proposes recovery actions and works with your team as it executes approved changes.

Technical and engineering support

L2 assistance covers configuration, logs, worker behaviour, tasks, offsets, throughput and recovery. L3 assistance covers deeper connector analysis and engineering within the agreed scope. Root cause reviews and service reporting can be included in the support schedule.

Security and lifecycle

For OSO products, supported releases and patch obligations follow the published security and support policies. For third-party and custom plugins, agree vulnerability triage, mitigation, dependency updates, release testing, backports and any maintained fork explicitly.

Coverage that fits the use case

Product subscriptions use their published support hours. Extended regional coverage, weekends, holidays and continuous 24-hour support can be arranged under a separate schedule. Define severity, response targets and escalation contacts against business impact.

Responsibilities and access

No standing production access is required for the standard remote model. The customer owns its infrastructure, source systems and production changes. Hands-on hypercare can be separately scoped. Initial response means acknowledgement and the start of investigation, not guaranteed restoration.

Platform support, connector support and code maintenance are recorded explicitly in the applicable agreement.

Build a platform teams can use

Start with a working integration, establish operational controls, and turn the proven pattern into reusable services for other teams.

Architecture and implementation

Design Kafka Connect worker groups for workload and security isolation. Establish topic naming, access boundaries, resource sizing, internal state storage and recovery. Implement on Kubernetes with Strimzi, including AKS and Azure Event Hubs designs where appropriate.

GitOps and Backstage self-service

Package approved plugins and deploy through reviewed Argo CD workflows. Build Backstage templates for connector instances or worker groups, with ownership metadata, configuration validation and secret references. Agree approval gates and lifecycle actions with the platform team.

Integration and event contracts

Use representative source samples to define event contracts and transformations. Test schema changes, replay, rate limits, error handling, reconciliation and sink side effects. Synthetic workloads help validate behaviour before production rollout.

Custom connector engineering

Build JVM source or sink connectors for systems that lack a suitable plugin. Scope authentication, pagination or streaming, state handling, delivery semantics, testing, packaging and documentation. Agree source ownership and ongoing security maintenance alongside development.

Operational readiness and enablement

Prepare runbooks, monitoring and escalation paths. Rehearse failure and recovery scenarios, agree production acceptance criteria, and hand over the deployment pattern to your engineering and operations teams.

Delivery services are separately scoped. Capacity, system access, source readiness and acceptance criteria shape the delivery plan.

Prove the first flow. Plan the next.

A short discovery and architecture review turns the service catalogue into a supported estate, delivery plan and operating agreement.

1. Review requirements and the estate

Confirm source and sink systems, versions, event rates, data sensitivity, regions, retention and business impact. Review the existing platform and agree which connectors and environments need support.

2. Validate the platform and first integration

Prove authentication, plugin packaging, worker recovery, offsets, retries and duplicate handling. For Azure Event Hubs, verify the required Kafka behaviour, compacted state hubs and namespace quotas with the actual versions and tier.

3. Agree support and production readiness

Record coverage, incident priorities, response clocks, escalation and change ownership. Define connector code-fix and security responsibilities. Establish a supported version window and the evidence required for production acceptance.

4. Expand through reusable patterns

Add approved connector families and self-service templates as use cases develop. Review capacity, isolation and operational outcomes as the supported estate grows.

Archive, replay and vendor portability

Long-term history needs a durable archive and a tested replay design. Assess object storage, capture and Kafka Backup against retention, access and recovery requirements. Rehearse restore and migration with plugin configuration, offsets, source cursors and sink side effects in mind.